Task1

1.byte、short、int、long、char、boolean、flaot、double。
2.byte:1Byte,$-128 \sim 127$;
short:2**Byte,$2^{15} \sim 2^{15}-1$;3.2E4
int:4Byte,$-2^{31} \sim 2^{31}-1$; 2.1E9
long:8Byte,$-2^{63} \sim 2^{63}-1$;9E18

3.基本类型　　引用类型
(1)存的是值　　存的是对象的地址
(2)内存在栈　　存的地址在栈，而对象在堆
(3)赋值后不干扰 复制后互相干扰
(4)比较的是值　比较的是地址是不是一样

4.发生隐式转换。

5.b=52;
char隐式转换为int类型为48，再与a相加。
6.运行结果为
13
6 8

7.计算机采用二进制储存，当数据太大超出范围，相加会进1，使符号位变为1，而符号位为1表示负数。

8.因为二进制存储的原因，所以小数位也是 2 的负多少次方，所以在存储某些数据时是无限的，但是 float 位数有限截取了部分所以不准确

9.包装类是基本类型对应的对象，存储在堆中；
缓存池是提前存在堆中的对象，可以直接引用，引用相同时对应的地址一样。

Task2

2.1

```
boolean isLeapYear(int year){
    if(year%100==0){
        if(year%400==0){
            return true;
        }else{
            return false;
```

```
        }
    }else{
        if(year%4==0){
            return true;
        }else{
            return false;
        }
    }
}
```

1.switch-case:在有限个，个数少，可以举出来的时候；
if-else：在一个范围，个数多，不好列举

2.不同
它们的适用场景不一样，不能说谁更简单，而且他们本质不一样，一个不会转变为另一个

2.1

```
void print(int n){
    for(int i=1;i<=(n+1)/2;i++){
        for(int j=i;j<=(n-1)/2;j++){
            System.out.print(" ");
        }
            for(int j=1;j<=2*i-1;j++){
            if(j==1||j==2*i-1){
                System.out.print("*");
            }else{
                System.out.print(" ");
            }
        }
        System.out.println();
    }
    for(int i=1;i<=(n-1)/2;i++){
        for(int j=1;j<=i;j++){
            System.out.print(" ");
        }
                for(int j=n;j>2*i;j--){
            if(j==n||j==2*i+1){
                System.out.print("*");
            }else{
                System.out.print(" ");
            }
        }
        System.out.println();
    }
}
```

2.3

递归

```
int fibonacci(int n){
    if(n==1||n==2){
        return 1;
    }else if(n>2){
        return fibonacci(n-1)+fibonacci(n-2);
    }else{
        return -1;
    }效率低；占用大量内存，可能溢出
}
```

迭代

```
int fibonacci(int n){
    if(n==1||n==2){
        return 1;
    }else if(n>2){
        int a=1;
        int b=1;
        int sum=0;
        for(int i=3;i<=n;i++){
            sum=a+b;
            a=b;
            b=sum;
        }
        return sum;
    }else{
        return -1;
    }
}
```

```
//第n-1次是把A放到C上，第n次一样，那么就有一种递推关系，把第n次放到B上再把A放到C上，最后把B放到C上，但是要把n-1次放到B上，所以放的位置不是固定的所以用 形参代表；
//方法是表示交换两个的位置
//分别输入A,C表示开始位置，目标位置
void hanoi(int n,char a,char b){

       char c=(char)('A'+'B'+'C'-a-b);//获得另外一个字符
    while(true){
        if(n == 1){//如果是1，直接传到目标位置，并结束本次方法
            System.out.println(a+"->"+b);
            return;
        }else{//开始将开始位置的放到另外一个位置，然后把最后一个放到目标位置
            n--;//并打印出路径；
            hanoi(n, a,c);
            System.out.println(a+"->"+b);
            char temp=a;//移动后开始位置变到了另一个位置，末位置没有变
            a=c;
```

```
            c=temp;//交换了之后再次循环


        }
    }



}


void hanoi(int n,char a,char b){

       char c=(char)('A'+'B'+'C'-a-b);

        if(n == 1){
            System.out.println(a+"->"+b);
            return;
        }else{
            n--;
            hanoi(n, a,c);//先把上面的n-1个放到B
            System.out.println(a+"->"+b);
            hanoi(n,c,b);//在把B放到C上
        }

}
```

2.3

1.递归是方法反复调用自己或者方法之间来回调用；
迭代是通过循环来重复执行代码；
2.相比迭代，递归更占内存，有风险；递归速率更慢
3.能
会占用更多内存，有风险；速率慢

Task3

1.Error是系统级的问题，在JVM层的问题，难以恢复----不处理
Exception是程序层的问题，一般可以解决---捕获并处理
2.(1)checked exception是编译器强制检查，必须捕获或抛出，否则报错；
unchecked exception是编译器不处理的异常，可以编译，在运行时报错；
(2)checked exception是由于运行环境无法确定安全；
unchecked exception是由于代码问题。
3.

```
package Task3;
```

```java
import java.io.*;

public class Test {
    public static void main (String[] args)  {
        File file = new File("data.txt");
        String len;
        int sum = 0;
        int count = 0;
        try (BufferedReader bis = new BufferedReader(new FileReader(method(file)))) {
            while ((len = bis.readLine()) != null) {
                sum += Integer.parseInt(len);
                count++;
            }//文件如果不存在结果length（）为0，所以Empty在下面，如果文件存在为零不会报不存在的异常
        }  catch (NumberFormatException e) {
            e.printStackTrace();
        } catch (FileNotFoundException e) {
            e.printStackTrace();
        } catch (EmptyFileException e) {
            e.printStackTrace();
        } catch (IOException e) {
            e.printStackTrace();
        }
            //如果程序异常被捕获，count没有自增，count为0，分母为0，会报错
            if (count != 0) {
                System.out.println((double) sum / count);
            }
        System.out.println("程序结束");
    }

    public static File method(File file) throws EmptyFileException{
        if((file.length())==0){
            throw new EmptyFileException("文件为空");
        }
        return file;
    }
}
```

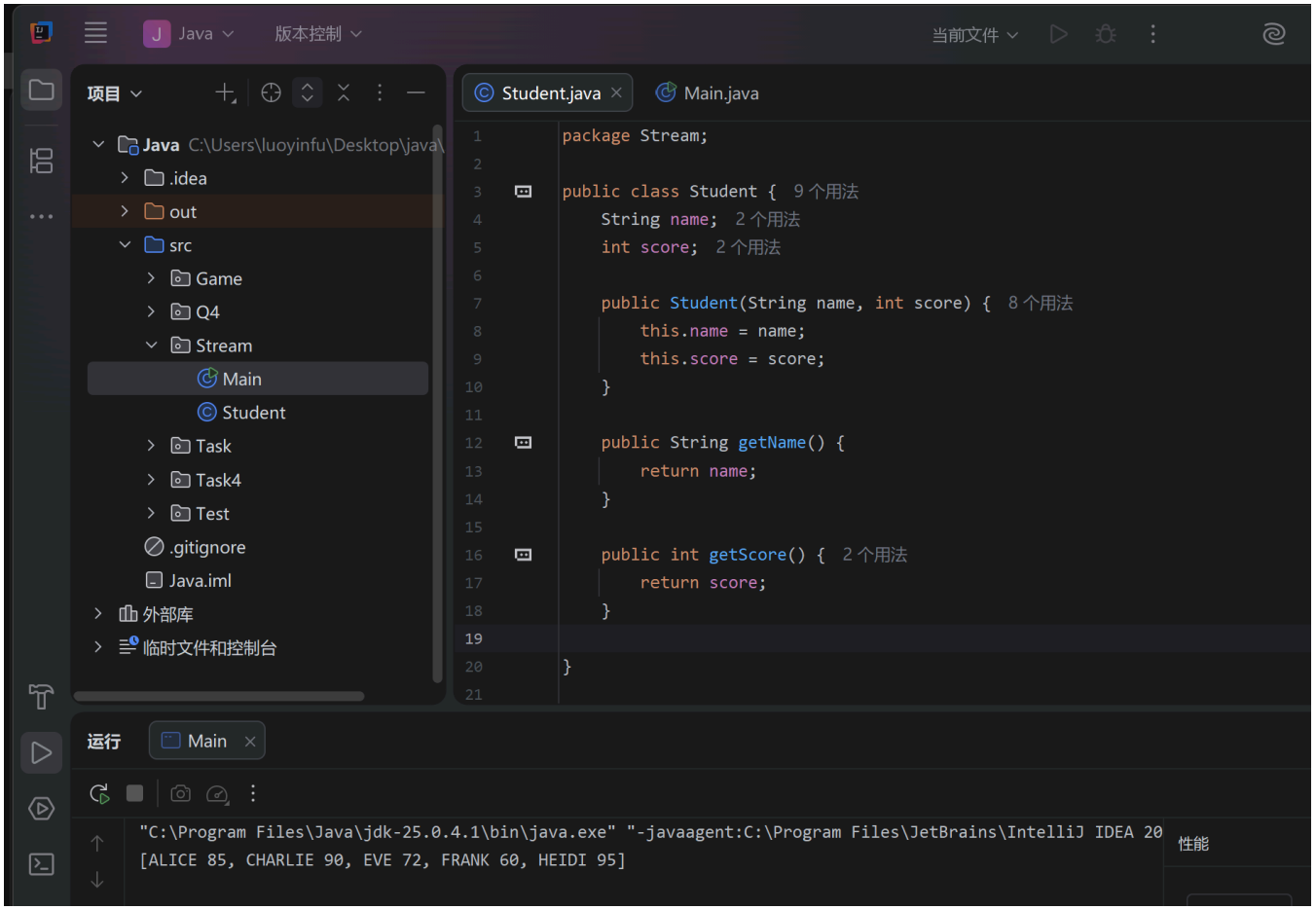
```java
package Task3;

public class EmptyFileException extends Exception {
    public EmptyFileException() {
    }

    public EmptyFileException(String message) {
        super(message);
    }
}
```

Task4

1.String;
因为流没有重写toString所以是继承Object的toString，而Object的toString打印的是类型名+"@"+哈希十六进制码
2.



Add

:
1.既然正数相加变负是溢出，那么负数是怎么存储的呢?
正数取反加
通过
ai
2.throw
和抓取的过程是怎么样的为什么创建了对象就可以抓取

执行过程（5 步）：

1. main 调用 `divide(10, 0)`
2. `divide` 里发现 b==0，执行 `throw new ArithmeticException(...)` → **创建一个异常对象，扔出去**
3. 这个对象 "往上抛"：从 divide **一路返回**到 main 的调用处
4. JVM 找有没有 `catch` 能接：**按类型匹配**，`ArithmeticException` 正好被 `catch (ArithmeticException e)` 接住
5. 接住后，程序**不崩溃**，继续往下执行

为什么 catch 能接住?

3.

printStatcTrace

打印结果是什么及什么意思

打印内容 = 每一层的调用者

plaintext

```
java.lang.NullPointerException       ← 异常类型
    at A.methodC(A.java:10)          ← methodC 出错的地方
    at B.methodB(B.java:20)          ← 是 methodB 调用了 methodC
    at Main.main(Main.java:30)       ← 是 main 调用了 methodB
```